

LAB REPORT

Course Title: Computer Graphics Lab
Course Code: CSE-4110

Topic: Interactive Sunset Kite Scene Simulation Using
OpenGL and GLUT

Submitted To:
Dr. Md Manjur Ahmed
Associate Professor
Department of Computer Science
and Engineering
University of Barishal

Submitted By:
Name: Md. Faysal Ahamed
Roll: 21-CSE-040
Name: Farjana Haque Khan
Roll: 21-CSE-006

Department: Computer Science and Engineering, University of Barishal

Submission Date: September 26, 2026

1 Abstract

This project presents an interactive 2D simulation of a sunset environment developed using OpenGL and GLUT in C++. The scene depicts a dynamic environment containing a house, trees, floating clouds, and a boy flying a kite. Central to the simulation is a time-of-day mechanic where the sun slowly sets, dynamically altering the ambient lighting, sky colours, and triggering specific event states—such as the boy autonomously walking home and the background music stopping. The project implements continuous animations (kite wobbling, cloud drifting, leg movement) and user interactivity to control the sunset speed and kite elevation. This report outlines the practical implementation of 2D geometric modelling, colour interpolation, state-based animation, and multimedia audio integration in computer graphics.

2 Introduction

Computer Graphics involves the creation, manipulation, and display of visual content through computational algorithms. OpenGL is a widely adopted standard for rendering 2D and 3D vector graphics. In this project, a cohesive, interactive 2D scene is programmed utilizing OpenGL alongside the GLUT utility toolkit. The simulation revolves around a neighbourhood setting where environmental conditions change smoothly over time. A core focus was placed on linking visual parameters (like ‘timeOfDay’) to lighting and behaviour. By utilizing simple geometric primitives—such as polygons, triangles, and lines—complex objects like clouds, trees, and a human figure are constructed. Additionally, the Windows Multimedia API (‘mmsystem.h’) is employed to provide a continuous background soundscape that reacts logically to the scene’s progression.

3 Project Objectives

The primary objectives of this project are:

- To construct a 2D scenic environment using OpenGL primitives (GL_QUADS, GL_TRIANGLE_FAN, GL_LINES, GL_LINE_STRIP).
- To implement a dynamic, mathematically interpolated day-to-night transition system affecting sky and object colours.
- To create continuous autonomous animations using trigonometric functions (kite wobbling, cloud drifting, walking cycle).
- To implement state-machine logic where environmental triggers (sun position) cause character behaviour changes (walking home).
- To integrate keyboard-based interactions allowing users to manipulate scene variables (kite height, time acceleration).
- To load and control background audio using the Windows Multimedia system.

4 Software and Libraries Used

The following software tools and libraries were used to develop the simulation:

Table 1: Software and Libraries Setup

Software / Library	Type	Purpose
C++	Language	Core programming language used for the entire project.
OpenGL (GL)	Library	Low-level 2D graphics rendering primitives and colour management.
GLUT	Toolkit	Utility for window creation, event loop, string rendering, and callbacks (<code>glutDisplayFunc</code> , <code>glutTimerFunc</code>).
Windows (<code>mmsystem.h</code>)	API	Audio playback control using <code>mciSendStringA</code> to load and loop the background MP3 file.
Standard C++ Library	Library	Mathematical calculations (<code>std::sin</code> , <code>std::cos</code>) and string manipulation for file paths.
IDE / Compiler	Tooling	Used for writing, compiling (linking <code>-lopengl32</code> , <code>-lglut32</code> , <code>-lwinmm</code>), and debugging.

5 System Design and Methodology

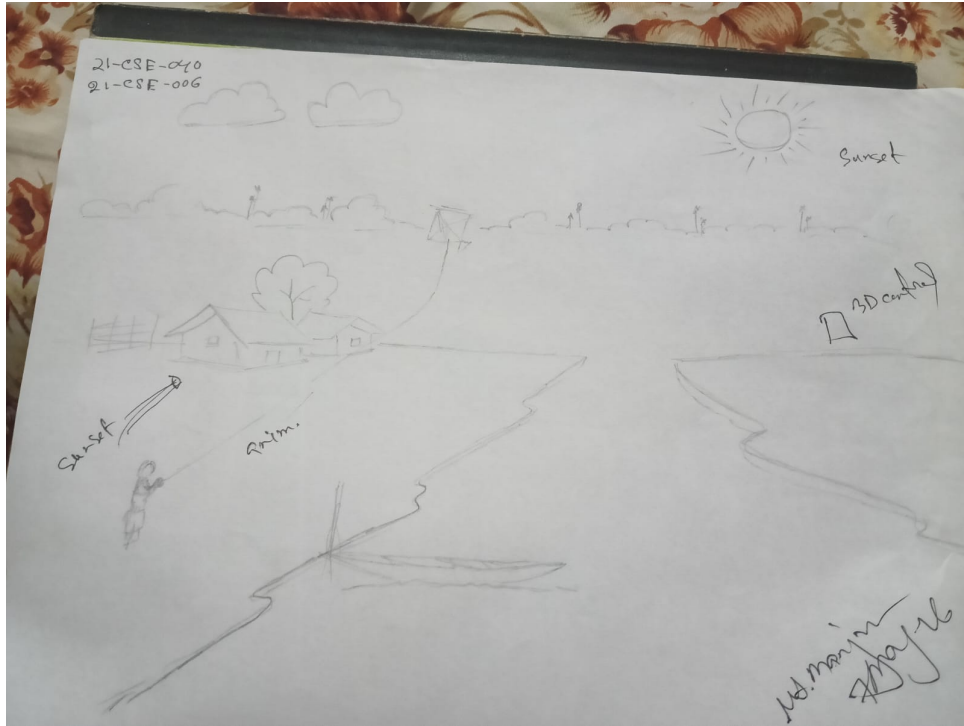


Figure 1: Hand-drawn conceptual diagram of the Sunset Kite Simulation

6 Scene Description

The scene uses a layered approach, drawing objects from the background to the foreground.

6.1 Sky and Sun

The background sky relies on dynamic colour interpolation. The `timeOfDay` variable ranges from 1.0 (day) to 0.0 (night). The `skyColor()` function calculates RGB values based on this range, transitioning from bright blue to warm orange, and finally to dark blue/black. The sun is drawn as a yellow circle using `GL_TRIANGLE_FAN`, and its brightness scales with the time of day.

6.2 Ground and House

The ground is constructed using layered `GL_QUADS`. Its colour is dynamically multiplied by the `timeOfDay` scalar to simulate darkening shadows as evening approaches. The house is constructed statically at the left side of the screen using quads for the base/door and a `GL_TRIANGLES` primitive for the roof.

6.3 Trees and Clouds

- **Trees:** Two trees are placed in the scene. The trunks are drawn with `GL_QUADS`, while the foliage consists of five overlapping green circles to create an organic, fluffy shape.

- **Clouds:** Rendered via the `cloud()` function, they use a three-layered shadow, body, and highlight system. Like the ground, the cloud’s ambient brightness dims as the sun sets.

6.4 The Boy and Kite

The boy is created using lines for the legs, a quad for the torso, and a circle for the head. The kite features a diamond shape (`GL_QUADS` with distinct colour halves) and a string attached to the boy drawn via `GL_LINE_STRIP`.

7 Animation Details

7.1 Day-to-Night Transition

The core simulation driver is the descending sun. Every timer tick, the `sunY` coordinate decreases. The `timeOfDay` normalisation is computed as:

$$\text{timeOfDay} = \frac{\text{sunY} - 160.0}{520.0 - 160.0}$$

This scalar is passed to almost every drawing function to uniformly dim the scene.

7.2 Kite Animation

The kite is designed to float organically. The angle (`kiteAngle`) oscillates back and forth using a directional multiplier (`kiteDir`). Its exact X and Y coordinates are plotted dynamically using trigonometric offsets from the boy’s position:

$$k_x = \text{boyX} + 60 + 40 \times \sin(\text{kiteAngle})$$

$$k_y = \text{boyY} + \text{kiteHeight} + 20 \times \cos(\text{kiteAngle})$$

7.3 Autonomous Walking Sequence

When the sun reaches a specific threshold (`sunY < 300.0f`), the boolean flag `boyWalking` is triggered.

- **Translation:** The boy’s X-coordinate shifts left towards `HOME_X`.
- **Leg Movement:** A sinusoidal offset is applied to the bottom vertices of the leg lines using `std::sin(animTimer * 0.25f)`, giving the illusion of a walking stride.

Once the boy reaches the house, the `boyInside` flag is set to true, making the boy and kite disappear from the screen, simulating him going indoors.

7.4 Audio Management

The `mcisendstringA` function from the Windows API handles audio. The path to ‘preview.mp3’ is dynamically resolved based on the executable’s directory. Audio plays on loop initially but is programmed to automatically stop when the sun falls below the horizon line (`sunY ≤ 160.0f`).

8 Keyboard Controls

The user can influence the environment using the following keyboard bindings:

Table 2: Keyboard Interaction Blueprint

Key	Action
SPACE	Instantly speeds up the sunset by decreasing the Sun's Y position.
W / w	Increases the kite's height (max limit 360.0f).
S / s	Decreases the kite's height (min limit 40.0f).
R / r	Resets the entire scene (sun, boy, clouds) and restarts audio.
ESC (27)	Exits the simulation.

9 OpenGL Functions Used

9.1 Drawing Primitives

- `GL_QUADS`: Used for the house base, ground planes, tree trunks, boy's torso, and kite body.
- `GL_TRIANGLE_FAN`: Central to the `circle()` helper function, used for the sun, boy's head, tree leaves, and clouds.
- `GL_TRIANGLES`: Used specifically for the house's slanted roof.
- `GL_LINES` / `GL_LINE_STRIP`: Used for drawing the boy's legs and the kite string.

9.2 Transformations and Setup

- `glOrtho(0, W, 0, H, -1, 1)`: Establishes a 2D orthographic projection matching the 800x600 window size, allowing direct mapping of pixels to coordinates.
- `glLoadIdentity()`: Resets the coordinate system before drawing each frame.

9.3 Colour and Rasterization

- `glColor3f()`: Dynamically sets the RGB colour state for primitive drawing.
- `glClearColor()`: Manipulates the background clearing colour for the sky.
- `glRasterPos2f()` & `glutBitmapCharacter()`: Used to render control instructions and student IDs directly onto the viewing window.

10 Code Structure Overview

The software is logically partitioned into the following areas:

1. **Global Configurations:** Defines window boundaries, object coordinates, state flags, and the audio file location.
2. **Audio Management:** `buildAudioPath()`, `startAudio()`, and `stopAudio()` ensure robust handling of multimedia regardless of the execution directory.
3. **Geometry Helpers:** The `circle()` function prevents code duplication for rounded objects.
4. **Entity Renderers:** Modular functions like `cloud()`, `tree()`, `house()`, `boy()`, and `kite()` handle the drawing logic for individual components.
5. **Display Loop:** The `display()` function clears the buffer, sets the sky, calls entity renderers in back-to-front order, and overlays bitmap text.
6. **Timer Callback:** `timer()` fires every 16ms (~ 60 FPS), incrementing animation phases, checking collision/threshold logic, and invoking `glutPostRedisplay()`.
7. **Input Handling:** `keyboard()` intercepts keystrokes to modify variables or trigger `resetScene()`.

11 Results & Observations

Upon compiling and running the code, the following behaviours were verified:

- The scene rendered correctly with smooth 60 FPS performance.
- The sky and environmental shading transitioned seamlessly from bright daylight colours to dark hues based entirely on the sun's Y-coordinate.
- The MP3 file played correctly at launch and ceased immediately upon the sunset threshold being crossed.
- The kite exhibited a natural-looking sway, and adjusting its altitude using 'W' and 'S' worked flawlessly within the bounded limits.
- Once triggered by dusk, the boy's walking animation (translating X and oscillating legs) executed smoothly until he vanished "inside" the house, simultaneously removing the kite from the scene.
- The on-screen text dynamically updated to alert the user that "Night - Boy is inside the home!".



Figure 2: The simulation running in daytime, showing the boy flying the kite.

12 Discussion

This simulation efficiently demonstrates the core philosophy of procedural animation and state-based rendering. Rather than hard-coding frame-by-frame changes, mathematical formulas (such as linear interpolation for light dimming and sine waves for walking/wobbling) govern the visuals. This makes the animation infinitely smooth and easily adjustable.

A notable architectural success in this code is the robust handling of the background audio path using `GetModuleFileNameA`. This prevents the common "file not found" errors that occur when executing C++ programs from different directories or IDEs. Furthermore, bounding the kite's height prevents the visual from breaking, demonstrating good user-input sanitization.

13 Conclusion

This laboratory assignment successfully combines fundamental 2D graphics programming with multimedia integration and logic-based animation. By constructing a cohesive scene where environmental states (time of day) directly impact entity behaviours (lighting, character movement, audio), the simulation achieves a high degree of interactivity and polish. The project serves as an excellent foundational exercise in understanding the OpenGL rendering pipeline, the GLUT event loop, and state-machine logic in C++.



Figure 3: The simulation transitioned to night. The boy is going inside, and text updates accordingly.